MVPCOMPANION

From App Idea to MVP

The Complete Guide

A practical guide to turning an uncertain product idea into a focused, testable and buildable MVP specification.

Clarify before designing.
Model before listing features.
Scope before specifying.
Define learning before building.

A useful MVP is not simply a smaller feature list. It is the smallest coherent product that can deliver the core promise and test important assumptions.

Resource Guide · MVPCompanion V2.0 · October 2026

How to use this guide

This guide is for founders, product thinkers and developers who have an app idea and want to prepare more systematically before building. Use it as an orientation and working reference—not as proof that a market exists or that a product will succeed.

What you will work through

Part	Purpose
1. Understand the method	See how MVPCompanion moves from uncertainty toward a coherent specification.
2. Clarify the idea and problem	Separate the initial solution idea from the problem to be solved.
3. Define user and value	Identify a primary user, desired outcome and meaningful differentiation.
4. Model the product and jobs	Establish the product model, lifecycle and Core Jobs before scope.
5. Derive scope and experience	Select the smallest coherent solution and map its core journey.
6. Specify and learn	Define data, requirements, technical boundaries and learning signals.
7. Review readiness	Check coherence, traceability and unresolved questions.

The MVPCompanion architecture

The current method uses **15 Stages of Progressive Clarification**, challenged throughout by **five Cross-Cutting Lenses**: Desirability, Differentiation, Viability, Feasibility and Learning. Market Discovery and Business Model are cross-cutting workstreams, not extra stages.

Evidence / Reasoning uses four labels: **E — Evidence**, **H — Hypothesis**, **D — Derived** and **O — Open**. Traceability connects product reasoning to requirements and observable metrics. AI can support thinking; people remain responsible for consequential decisions.

Use this guide progressively. You do not need certainty at every stage. You do need to make important uncertainty visible and avoid silently turning assumptions into facts.

1. The 15-stage journey

Each stage answers a different question and produces an output that supports later work. The stages are a progression toward specificity—not a promise that early hypotheses are true.

# / Stage	Core question	Primary output
1. Idea	What could this be?	Idea Brief / Idea Hypothesis
2. Problem	What real problem exists?	Problem Statement + evidence / assumptions
3. User	Who has the problem?	Primary User Profile + target / non-target
4. Value	Why does solving it matter?	Value Proposition, benefits, USP, positioning
5. Product	What is the product fundamentally?	Product Mental Model + Entity Map
6. Context & Lifecycle	In what context does value arise, and how do relevant users/objects/processes evolve?	Context & Lifecycle Model
7. Core Jobs	What must users accomplish?	Prioritized Core Jobs
8. Scope	What is the smallest useful solution?	MVP Feature Map + Out-of-Scope List
9. Experience	How does the user receive the value?	Core User Flows + Screen Inventory
10. Information Architecture	How is the product organized?	IA + Navigation Model
11. Data	What information must exist?	MVP Entity/Data Model
12. Requirements	What exactly must the system do?	Functional Requirements + Acceptance Criteria
13. Learning	What must the MVP teach us?	Success Metrics + Learning Questions
14. Technical Boundaries	What technical conditions constrain the MVP?	High-Level Architecture + Technical Constraints
15. MVP Boundary	What exactly is the smallest coherent product?	Final MVP Scope + Exclusions + Boundary Statement

The stages are supported by the five lenses, Market Discovery, Business Model, the E/H/D/O discipline, Traceability, the Learning Loop, AI Companion support and human judgment.

2. Clarify the idea, problem and user

Stage 1 — Idea: capture the raw idea before polishing it. Record the trigger, context, intended user, expected benefit and major assumptions. Do not prematurely validate or reduce the idea to a feature list.

Quick exercise: the Idea Brief

- **Idea:** In one or two sentences, what might the product do?
- **Trigger:** What observation, frustration, request or opportunity prompted the idea?
- **Intended user:** Who might benefit? Keep this provisional.
- **Expected outcome:** What progress or accomplishment should become easier?
- **Major assumptions:** What would have to be true for this idea to matter?

Stage 2 — Problem: describe the difficulty independently of the proposed solution. Examine the situation, current behavior, frequency, consequences, alternatives and workarounds. A solution disguised as a problem makes it harder to evaluate whether the underlying need is real.

Problem statement starter

When [situation/context], [specific user or segment] struggles to [accomplish something] because [observed difficulty], leading to [consequence].

Treat this as a draft until supported. Do not insert a cause or consequence merely because it sounds plausible.

Stage 3 — User: select a primary user whose problem and context justify investigating a behavior change. Describe constraints, current workflow and alternatives. Identify secondary users separately and explicitly name non-target users for the initial product.

Ask yourself

- Who experiences the problem most directly, and in what situation?
- What do they do today—including manual workarounds?
- Who is not the initial target, and why?
- What evidence supports this segment choice? What remains uncertain?

Avoid treating a broad audience or demographic persona as proof of problem intensity or demand.

3. Define value and model the product

Stage 4 — Value: connect the problem and primary user to a meaningful outcome. State the value proposition without relying on a feature list. Consider benefits, alternatives, differentiation and the business assumptions that may matter.

Value proposition starter

> **For [primary user] who [problem/context], the product helps them [desired outcome] by [high-level value mechanism].**

Then challenge the statement: Is the outcome important? What do users do instead? Why might this proposition be preferable? Which claims are evidence-based, and which are hypotheses?

Jobs-to-be-Done (JTBD) may be used as a supporting analytical approach to understand user intent, progress and context. In MVPCompanion, **Job** is the canonical user-intent concept; JTBD is not an additional stage or mandatory artifact.

Stage 5 — Product: define what the product fundamentally is before designing screens. Identify the central entity, supporting entities, relationships and the distinction between durable objects and actions. Decide what information needs structured fields and what can remain free-form.

Product mental model worksheet

Prompt	Your notes
What is the central object the user creates, owns, manages or transforms?	
What supporting objects are necessary?	
How are those objects related?	
Which information is structured, and why?	
Which information can remain free-form?	
What is essential for the first coherent product?	

Stage 6 — Context & Lifecycle: identify meaningful contexts, states and transitions where they affect how value is created. Keep only states with operational meaning; do not add lifecycle complexity for completeness alone.

4. Prioritize Core Jobs and derive scope

Stage 7 — Core Jobs: identify what the primary user must accomplish to receive the promised value. A Job describes the user's intended accomplishment—not a screen, feature or technology.

For each candidate Job, capture

- Trigger or situation.
- What the user needs to do or decide.
- Minimum information needed.
- Desired result or outcome.
- Why it matters to the value proposition.

Prioritize a small set—often 3–7 where appropriate—and document uncertainty. The number is a guide, not a quota. Separate secondary Jobs from those selected for the MVP.

Stage 8 — Scope: derive candidate capabilities from prioritized Core Jobs. Group them into coherent areas, classify them and explicitly document exclusions. Challenge every proposed Must Have.

Scope class	Meaning
Must Have	Needed for the smallest coherent product to deliver its core promise or test a critical assumption.
Should Have	Valuable, but not essential to the initial coherent MVP.
Later	Potentially useful after further learning or development.
Not in MVP	Explicitly excluded from the current boundary.

The scope challenge

- Which prioritized Core Job does this capability support?
- What happens if we remove it?
- Does it depend on other capabilities or data?
- Is it required for the end-to-end journey or a justified learning objective?
- What evidence supports including it now?

Trace every included capability to a prioritized Core Job. If the connection is unclear, challenge the feature or document the unresolved rationale.

A smaller feature list is not automatically a better MVP. The goal is the smallest coherent solution that preserves the core promise and can generate meaningful learning.

5. Make the product concrete

Stage 9 — Experience: design a complete path from entry to first value and core outcome. Define onboarding, the first object created or imported, the happy path and essential empty, error and permission states. Create the screen inventory after the flows.

Stage 10 — Information Architecture: organize navigation and information according to the product mental model and Core Jobs. Every MVP capability should have a natural place; navigation should not simply mirror internal departments.

Stage 11 — Data: define the minimum implementable data model. List essential entities, fields, relationships, ownership boundaries and lifecycle fields where relevant. Distinguish manually entered, imported and computed data. Remove fields without a clear purpose.

Stage 12 — Requirements: specify observable system behavior so another person can understand what to build and how completion will be judged.

A useful requirement should make clear

- Purpose and the Core Job or capability it supports.
- Expected behavior, inputs and outputs.
- Validation and relevant states: empty, loading, success, error and disabled.
- Permissions and integration behavior where relevant.
- Dependencies and priority.
- Acceptance criteria that can be checked.

Example of an acceptance criterion

> **Given [defined starting condition], when [specific user action occurs], then [observable system result] should occur.**

This is a writing pattern, not a substitute for product-specific detail. Include examples and edge cases wherever a developer or tester would otherwise need to guess.

Do not confuse a screen description with a functional requirement. Specify behavior and outcomes, not just appearance.

6. Define learning and technical boundaries

Stage 13 — Learning: make the MVP a learning instrument. Identify the assumptions that matter, the questions that must be answered and the evidence that could change a decision.

Learning worksheet

Question	Record
What important hypothesis are we testing?	
What must we learn to make the next decision?	
What user behavior or result could be observed?	
What signal or metric could be relevant?	
What result would change our current view?	
What decision follows from the evidence?	

Choose metrics that connect to the product hypothesis. Depending on the product, relevant signals may include activation, a core value event, continued use, quality or willingness to pay. Avoid vanity metrics and do not assume that a metric proves causation.

Stage 14 — Technical Boundaries: define enough of the technical shape to estimate and build without solving every future architecture problem. Record platform assumptions, backend/API needs, storage, authentication, integrations, AI/service dependencies, risks and open questions.

Stage 15 — MVP Boundary: integrate the decisions from earlier stages. Confirm the primary user, core promise, prioritized Core Jobs, essential capabilities, end-to-end experience, minimum data, requirements, learning goals, technical constraints and explicit exclusions.

The MVP Boundary is a deliberate decision about what is inside and outside the current product. It is not simply the result of deleting a few features from a larger vision.

The integrated MVP Specification is the aggregated deliverable produced from the 15 stages—not an additional numbered stage.

7. Keep the reasoning connected

Traceability is a central quality check. It helps explain why a capability or requirement exists and how the product will produce observable results.

Problem → User → Outcome → Job → Capability → Requirement → Acceptance Criterion → Metric

Use the chain in both directions

- **Backward:** Start from a requirement and ask whether it supports a capability and Job, contributes to an outcome for the primary user, and relates to a meaningful problem.
- **Forward:** Start from an important requirement and ask how completion can be observed, which metric or signal matters, and what learning question it informs.

If a feature or requirement cannot be traced to a meaningful outcome, challenge it. If an important outcome has no supporting capability, the MVP may be incomplete.

The E/H/D/O discipline

Label	Use it for	Working rule
E — Evidence	Information with an identifiable supporting basis.	Record the source or basis; evidence can still be revised.
H — Hypothesis	A proposition requiring validation or further evidence.	Keep it visibly assumed until supported, rejected or revised.
D — Derived	A logical conclusion from documented inputs.	Show the inputs; do not use D as a substitute for E.
O — Open	An unresolved question, decision, dependency or uncertainty.	Do not silently turn an open item into a decision.

The learning loop

Evidence → Learning → Revised Hypothesis → Product Change

As new evidence appears, hypotheses may change and earlier decisions may need review. This is not a failure of the method; it is part of disciplined product learning.

8. Use AI as a companion—not a substitute

AI can help structure information, ask challenging questions, identify assumptions and contradictions, generate alternatives, derive implications from documented inputs, check traceability and prepare learning questions. It can also help draft requirements and acceptance criteria.

People remain responsible for consequential product decisions: choosing which problem matters, selecting the primary user, judging evidence, accepting or rejecting hypotheses, prioritizing trade-offs, deciding scope and accepting risk.

No AI-generated output automatically becomes a product decision. Review, revise or reject it explicitly.

A practical AI review prompt

“Review the material I provide for this MVPCompanion work step. Separate statements into Evidence, Hypothesis, Derived and Open. Do not invent facts or treat assumptions as evidence. Identify contradictions, missing information and unsupported claims. Check relevant links to the problem, primary user, outcome, Job, capability, requirement, acceptance criterion and metric where applicable. Suggest alternatives and questions, but label them as proposals. End with open issues and decisions that require human judgment.”

Stage Gates: decide whether it is sensible to progress

Status	Meaning
PASS	The stage is sufficiently coherent to progress.
PASS + OPEN	Progress is possible while explicit open items are carried forward.
HOLD	A critical unresolved issue prevents meaningful downstream work.
REVISIT	New evidence materially changes an earlier conclusion and requires rework.

A gate checks clarity, evidence, assumptions, traceability and open issues. Not every uncertainty blocks progress; the question is whether the uncertainty undermines the next stage.

9. Final MVP readiness checklist

Use this checklist as a final review—not as a numerical score or a guarantee of success. If an item is not ready, record what is missing and whether it blocks the next decision.

Strategy and user

- The problem can be explained without mentioning the proposed solution.
- The primary user and non-target users are explicit.
- The desired outcome and value proposition are clear.
- Alternatives and differentiation assumptions are visible.

Product and scope

- The product mental model and relevant lifecycle are coherent.
- Core Jobs are prioritized; every included capability supports a prioritized Job.
- Must Have, Should Have, Later and Not in MVP are distinguished.
- The product provides a coherent end-to-end journey and a clear first-value moment.

Specification and delivery

- Information Architecture and minimum data model are explicit.
- Critical requirements are observable and testable.
- Important requirements have acceptance criteria.
- Technical boundaries, dependencies and major risks are visible.

Learning and decision quality

- Important hypotheses and open questions are visible.
- Learning questions and relevant signals are defined.
- The AI role is bounded, if AI is included.
- Out-of-scope items are explicit.
- Another person could understand what to build and how to judge completion.

Readiness means the specification is sufficiently coherent for the next step. It does not mean the market has been validated or that the product is guaranteed to succeed.

Your next step

Start with the Idea Brief. Move forward by clarifying the problem, user and outcome before expanding into product design. Keep evidence and assumptions distinct, revisit decisions when meaningful new evidence appears, and use the final MVP Specification to align product, design and engineering.

MVPCompanion V2.0 · Resource Guide · October 2026

Method note: This guide is a practical introduction aligned to the MVPCompanion Method Baseline v0.2-R3 and the current 15-Stage Specification. It simplifies for orientation and does not replace the full method specification. Examples and worksheet prompts are illustrative; they are not claims of validated user or market evidence.